Universidade Estadual da Paraíba — UEPB

Centro de Ciências Exatas e Sociais Aplicadas — CCEA

Curso de Bacharelado em Ciência da Computação

Estrutura de Dados

Ordenação

Heron Aragão Monteiro

Slides parcialmente baseados no material do professor **Janderson J. B. Aguiar** (UEPB/CCEA)

Estrutura de Dados
Ordenação

- Ordenação
  - É fundamental para organizar informação
    - Motores de Busca
    - Sistemas Operacionais
  - Diversos algoritmos
    - Ordenação por comparação
      - Iterativos
      - Recursivos (divisão e conquista)
    - Ordenação em tempo linear

Estrutura de Dados
Ordenação

- Propriedades
  - Estabilidade de um algoritmo
    - Um algoritmo de classificação estável
      - Preserva a ordem relativa dos itens com valores repetidos
      - Faz mais sentido no contexto chave/registro
      - Exemplo
        - [nota/aluno] = [7/João], [8/Maria], [8/Carlos], [9,Pedro]
        - Algoritmo estável
          - [9,Pedro], [8/Maria], [8/Carlos], [7/João]
        - Algoritmo não estável
          - [9,Pedro], [8/Carlos], [8/Maria], [7/João]

Estrutura de Dados
Ordenação

- Propriedades
  - Algoritmo In place
    - Ordenação realizada no mesmo local em que os dados são armazenados
      - No máximo precisa de memoria extra de tamanho constante

Estrutura de Dados
Ordenação

- Bubble Sort
  - É um algoritmo de ordenação simples
  - Percorre repetidamente uma lista de elementos
    - Compara os pares adjacentes e troca-os se estiverem fora de ordem
  - Esse processo é repetido até que a lista esteja completamente ordenada

Estrutura de Dados
Ordenação

- Bubble Sort
  - Algoritmo

```
procedure bubbleSort(lista)
    tamanho <- tamanho da lista
    para i de 0 até tamanho - 1
        para j de 0 até tamanho - 1 - i
            se lista[j] > lista[j+1] então
                trocar lista[j] com lista[j+1]
            fim se
        fim para
    fim para
fim procedure
```

Estrutura de Dados
Ordenação

- Bubble Sort
  - Exemplo
    - Ordenar
      - [5, 2, 9, 1, 3]

Estrutura de Dados
Ordenação

- Bubble Sort
  - Complexidade
    - Considerando n = tamanho da lista
      - Complexidade de tempo: pior caso
        - Quando a lista está em ordem reversa
        - Bubble Sort precisa percorrer a lista completa n vezes
          - Para cada iteração realiza-se n - 1 comparações
        - O número total de comparações é (n - 1) + (n - 2) + ... + 1 = (n * (n - 1)) / 2
        - O número total de trocas também é aproximadamente o mesmo
        - Para cada comparação que resulta em uma troca, uma troca adicional é realizada
        - A complexidade de tempo do Bubble Sort é $O(n^2)$
      - Complexidade de espaço
        - O(1), pois a ordenação é in place

Estrutura de Dados
Ordenação

- Bubble Sort
  - Características
    - Fácil de implementar
    - Fácil de entender
    - In place
    - Estável
    - Pode ser útil para listas pequenas
    - Número alto de comparações e de trocas
    - Muito ruim na prática

Estrutura de Dados
Ordenação

- Selection Sort
  - Também é um algoritmo de ordenação simples
  - Divide a lista em duas partes
    - A primeira parte, à esquerda, representa a porção ordenada da lista
    - A segunda parte, à direita, representa a porção não ordenada
    - O algoritmo percorre a porção não ordenada, encontra o elemento mínimo e o coloca no final da porção ordenada
    - Esse processo é repetido até que a lista esteja completamente ordenada

Estrutura de Dados
Ordenação

- Selection Sort
  - Algoritmo

```
procedure selectionSort(lista)
    tamanho <- tamanho da lista
    para i de 0 até tamanho - 1
        índice_minimo <- i
        para j de i+1 até tamanho - 1
            se lista[j] < lista[índice_minimo] então
                índice_minimo <- j
            fim se
        fim para
        trocar lista[i] com lista[índice_minimo]
    fim para
fim procedure
```

Estrutura de Dados
Ordenação

- Selection Sort
  - Exemplo
    - Ordenar
      - [5, 2, 9, 1, 3]

Estrutura de Dados
Ordenação

- Selection Sort
  - Complexidade
    - Considerando n = tamanho da lista
      - Complexidade de tempo
        - O algoritmo possui duas operações principais
          - Encontrar o mínimo em uma porção não ordenada
          - Realizar a troca com o elemento da porção ordenada
        - Para cada iteração externa, o número de comparações internas diminui
          - Começando com n - 1 comparações na primeira iteração, n - 2 comparações na segunda iteração e assim por diante
        - O número total de comparações é (n - 1) + (n - 2) + ... + 1 = (n * (n - 1)) / 2
        - O número total de trocas é no máximo n – 1
          - O menor elemento é colocado na porção ordenada a cada iteração
        - Complexidade de tempo do Selection Sort é $O(n^2)$, no pior caso
      - Complexidade de espaço
        - O(1), pois a ordenação é in place

Estrutura de Dados
Ordenação

- Selection Sort
  - Características
    - Fácil de implementar
    - Fácil de entender
    - In place
    - Estável
      - Depende de como é feita a troca
    - Pode ser útil para listas pequenas
    - Número alto de comparações e de trocas

Estrutura de Dados
Ordenação

- Insertion Sort
  - Também é um algoritmo de ordenação simples
  - Divide a lista em duas partes
    - A primeira parte, à esquerda, representa a porção ordenada da lista
    - A segunda parte, à direita, representa a porção não ordenada
    - O algoritmo percorre a porção não ordenada, **seleciona um elemento** e o insere na posição correta na porção ordenada
    - Esse processo é repetido até que a lista esteja completamente ordenada

Estrutura de Dados
Ordenação

- Insertion Sort
  - Algoritmo

```
procedure insertionSort(lista)
    tamanho <- tamanho da lista
    para i de 1 até tamanho - 1
        valor_atual <- lista[i]
        j <- i - 1
        enquanto j >= 0 e lista[j] > valor_atual
            lista[j+1] <- lista[j]
            j <- j - 1
        fim enquanto
        lista[j+1] <- valor_atual
    fim para
fim procedure
```

Estrutura de Dados
Ordenação

- Insertion Sort
  - Exemplo
    - Ordenar
      - [5, 2, 9, 1, 3]

Estrutura de Dados
Ordenação

- Insertion Sort
  - Complexidade
    - Considerando n = tamanho da lista
      - Complexidade de tempo
        - O algoritmo possui duas operações principais
          - A comparação
          - O deslocamento dos elementos
        - Para cada elemento a partir do segundo, o algoritmo compara o elemento com os elementos anteriores na porção ordenada e, se necessário, desloca esses elementos uma posição para a direita para abrir espaço para a inserção
        - No pior caso, quando a lista está em ordem reversa, o algoritmo precisa percorrer a lista completa para cada elemento
        - O número total de comparações é (n - 1) + (n - 2) + ... + 1 = (n * (n - 1)) / 2
        - O número total de deslocamentos é aproximadamente o mesmo que o número de comparações no pior caso
        - A complexidade de tempo do Insertion Sort é $O(n^2)$ no pior caso
      - Complexidade de espaço
        - O(1), pois a ordenação é in place

Estrutura de Dados
Ordenação

- Insertion Sort
  - Características
    - Fácil de implementar
    - Fácil de entender
    - In place
    - Estável
    - Muita escrita (troca) e menos comparações
    - Pode ser útil para listas pequenas

Estrutura de Dados
Ordenação

- Algumas considerações
  - Questões de Implementação
    - Como desenvolver/adaptar os algoritmos de ordenação vistos para ordenar tipos genéricos?
    - É preciso trabalhar com um tipo T

BUBBLE-SORT(A)
SELECTION-SORT(A)
INSERTION-SORT(A)

A é uma lista de itens comparáveis (**Comparable**)

- Em vez disto:
  A[j] > A[j+1]
- Isto:
  A[j].compareTo(A[j+1]) > 0

Estrutura de Dados
Ordenação

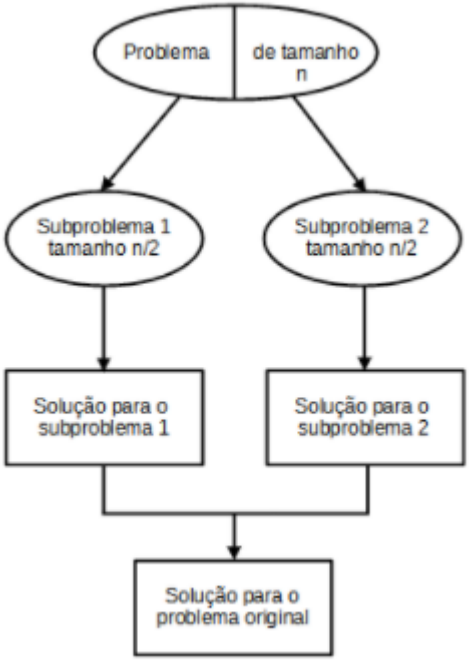
- Algumas considerações
  - Os algoritmos vistos são focados em uma varredura linear da entrada
    - Pior caso: O(n2)
  - Outra estratégia de ordenação
    - Divisão e conquista

Estrutura de Dados
Ordenação

- Divisão e conquista
  - Provavelmente a técnica de design de algoritmo mais conhecida
  - Funcionamento geral
    - Um problema é dividido em vários subproblemas do mesmo tipo
      - Idealmente de tamanhos aproximadamente iguais
    - Os subproblemas são resolvidos
      - Normalmente de forma recursiva
    - As soluções dos subproblemas, se necessário, são combinadas para obter uma solução para o problema original

Estrutura de Dados
Ordenação

- Divisão e conquista
  - Serão considerados apenas algoritmos sequenciais
    - Mas a técnica é ideal para cálculos paralelos
      - Cada subproblema pode ser resolvido simultaneamente por seu próprio processador



Estrutura de Dados
Ordenação

- Divisão e conquista
  - Recorrência geral da divisão e conquista
    - Para o caso mais típico de divisão e conquista
      - Instância de um problema de tamanho n é dividida em duas instâncias de tamanho n/2
    - Generalizando
      - Uma instância de tamanho n pode ser dividida em b instâncias de tamanho n/b
      - Sendo resolvidas a dessas instâncias
        - Sendo a e b são constantes, onde $a \geq 1$ e $b > 1$
      - Para simplificar a análise é assumido que n é uma potência de b
    - Tempo de execução T(n) poder ser recursivamente assim obtido
      - $T(n) = aT(n/b) + f(n)$
        - Onde
          - T(n/b) representa o tempo de dividir uma instância em instâncias de tamanho n/b
          - f(n) representa o tempo para combinar as soluções

Estrutura de Dados
Ordenação

- Merge sort
  - Exemplo de uma aplicação bem-sucedida da técnica de dividir e conquistar
    - Manipula seus elementos de entrada de acordo com sua posição no vetor
    - Algoritmo estável
    - Classifica um determinado vetor $A[0..n-1]$
      - Divide em duas partes $A[0..\ n/2-1]$ e $A[\ n/2\ ..n-1]$
      - In-place
      - Classifica cada uma deles recursivamente
      - Mescla os dois vetores classificados menores em um único vetor classificado

Estrutura de Dados
Ordenação

- Merge sort
  - Algoritmo

```
procedure mergeSort(lista)
    se tamanho da lista <= 1 então
        retornar lista
    senão
        meio <- tamanho da lista / 2
        lista_esquerda <- mergeSort(primeira metade da lista)
        lista_direita <- mergeSort(segunda metade da lista)
        retornar merge(lista_esquerda, lista_direita)
    fim se
fim procedure
```

Estrutura de Dados
Ordenação

- Merge sort
  - Algoritmo

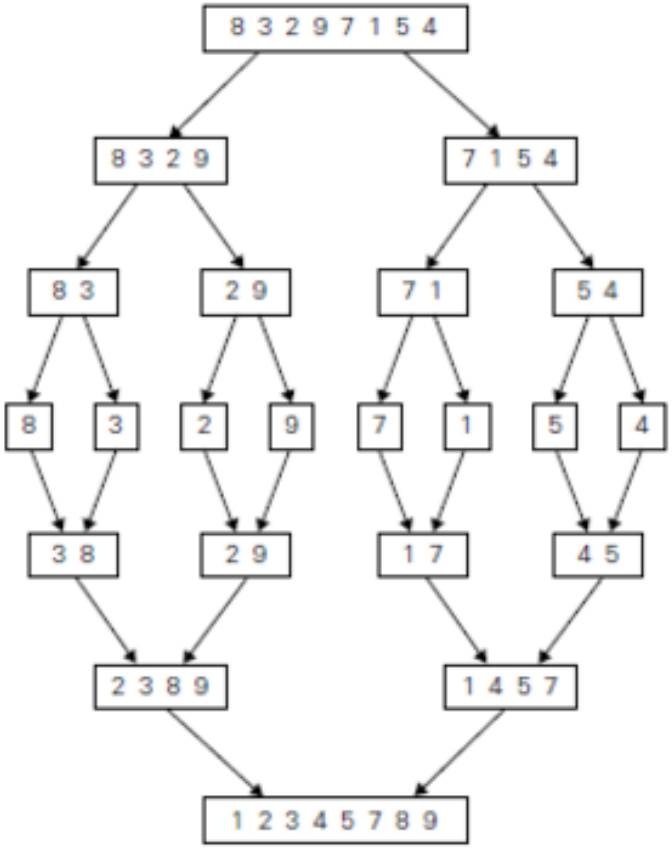
```
procedure merge(lista_esquerda, lista_direita)
    resultado <- lista vazia
    enquanto lista_esquerda não está vazia e lista_direita não está vazia
        se primeiro elemento de lista_esquerda <= primeiro elemento de lista
            adicionar primeiro elemento de lista_esquerda ao resultado
            remover primeiro elemento de lista_esquerda
        senão
            adicionar primeiro elemento de lista_direita ao resultado
            remover primeiro elemento de lista_direita
        fim se
    fim enquanto

    adicionar elementos restantes de lista_esquerda ao resultado
    adicionar elementos restantes de lista_direita ao resultado

    retornar resultado
fim procedure
```

Estrutura de Dados
Ordenação

- Merge sort
  - Exemplo



Estrutura de Dados
Ordenação

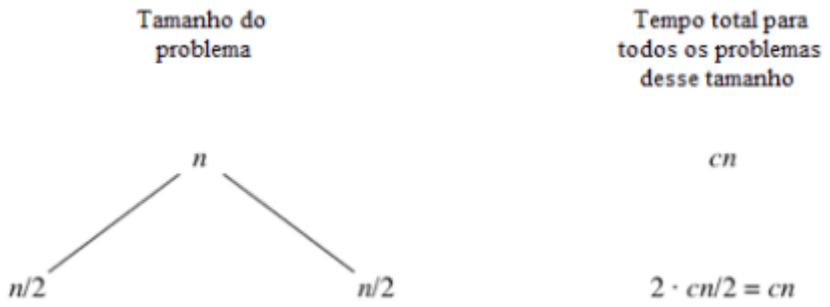
- Merge sort
  - Complexidade
    - Análise da complexidade de tempo
      - Função Merge
        - Cada comparação leva um tempo constante
        - Cada elemento é copiado para o vetor A exatamente uma vez
        - Cada linha de código é executada um número constante de vezes
          - Assumindo que o vetor A[p..q] tem n elementos
          - Tempo de execução para intercalar será O(n)

Estrutura de Dados
Ordenação

- Merge sort
  - Complexidade
    - Análise da complexidade de tempo
      - Função Mergesort
        - O passo da divisão
          - Calcula o ponto médio do vetor A no tempo $\Theta(1)$
          - Copia os elementos do vetor A nos subvetores B e C no tempo $\Theta(n)$
        - O passo da combinação intercala, como visto, n elementos no tempo $O(n)$
          - Os passos da divisão e combinação levam um tempo cn
            - onde c é uma constante

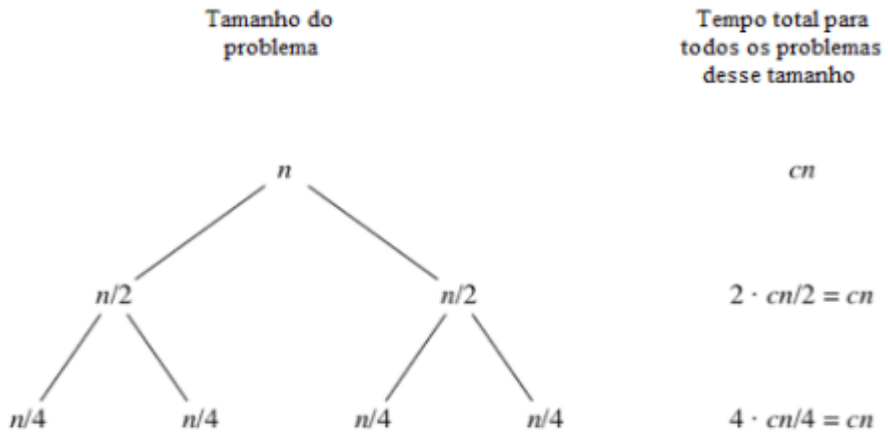
Estrutura de Dados
Ordenação

- Merge sort
  - Complexidade
    - Análise da complexidade de tempo
      - Função Mergesort
        - O passo da conquista
          - Assumindo n sempre par
          - Para duas chamadas



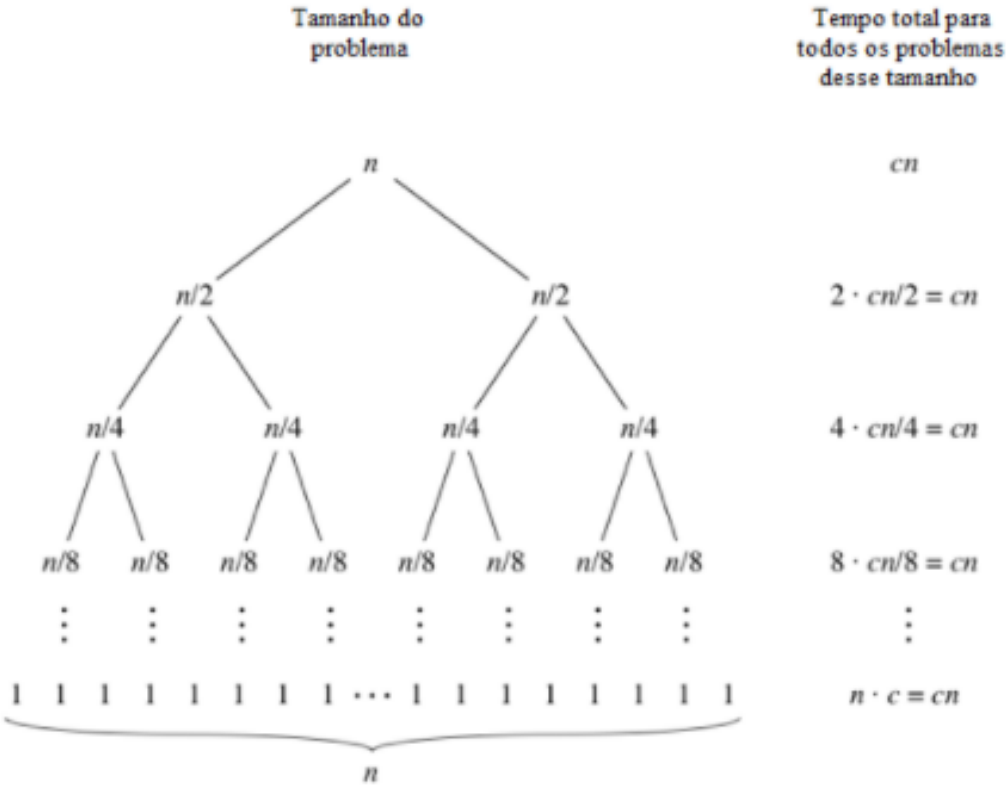
Estrutura de Dados
Ordenação

- Merge sort
  - Complexidade
    - Análise da complexidade de tempo
      - Função Mergesort
        - O passo da conquista
          - Para o próximo nível



Estrutura de Dados
Ordenação

- Merge sort
  - Complexidade
    - Análise da complexidade de tempo
      - Função Mergesort
        - O passo da conquista
          - Generalizando



Estrutura de Dados
Ordenação

- Merge sort
  - Complexidade
    - Análise da complexidade de tempo
      - Função Mergesort
    - O passo da conquista
      - O tempo total será a soma dos tempos de todos os níveis
        - Havendo l níveis
          - Tempo da conquista
            - $Tc(n) = l \cdot cn$
        - A quantidade de níveis será dada por
          - $l = \log_2 n+1$
        - Logo
          - $Tc(n) = (\log_2 n+1)\, cn$
          - $Tc(n) = cn\, (\log_2 n) + cn$
          - $O(n \log_2 n)$
    - Complexidade de tempo do algoritmo
          - $O(n \log_2 n)$

Estrutura de Dados
Ordenação

- Merge sort
  - Complexidade
    - Análise da complexidade de espaço
      - O algoritmo não trabalha localmente
      - Complexidade de espaço
        - O(n)
        - para fazer uma cópia de todos os elementos do vetor original

Estrutura de Dados
Ordenação

- Merge sort
  - Características
    - Boa performance: O(n log n)
    - Não é in place
    - Stable
      - Mas há variações de implementações não stable

Estrutura de Dados
Ordenação

- Quick sort
  - Divide seus elementos de acordo com o valor de um elemento pivô
    - Lembrando
      - Mergesort
        - Divide seus elementos de entrada de acordo com sua posição no vetor
  - Utiliza a ideia de partição
    - Todos os elementos à esquerda de algum elemento A[s] são menores ou iguais a A[s]
    - Todos os elementos à direita de A[s] são maiores que ou igual a ele
      - A[s] estará na sua posição final

$$\underbrace{A[0]\ldots A[s-1]}_{\leq A[s]}\ A[s]\ \underbrace{A[s+1]\ldots A[n-1]}_{\geq A[s]}$$

Estrutura de Dados
Ordenação

- Quick sort
  - A ordenação continua particionando os dois subvetores esquerdo e direito, independentemente
    - Todo o trabalho acontece na fase de divisão
    - Não é necessário combinar os subproblemas
  - Trabalha in-place
  - Não estável

Estrutura de Dados
Ordenação

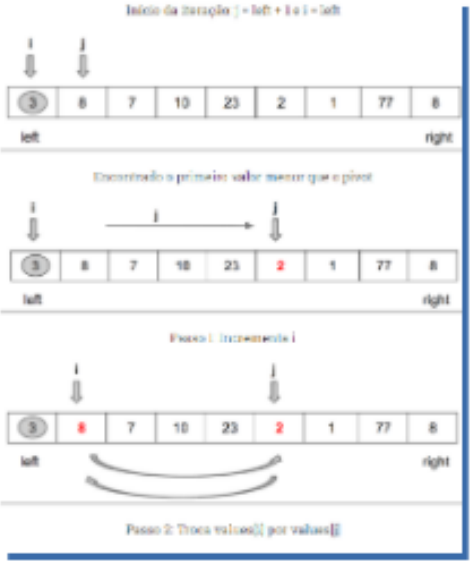
- QuickSort
  - ALGORITMO Quicksort(A[l..r])
    - //Classifica um subvetor por quicksort
    - //Entrada: Subvetor do vetor A[0..n − 1],
    - // definido por seus índices esquerdo e direito l e r
    - //Saída: Subvetor A[l..r] classificado
    - se l < r
      - s ←Particiona(A[l..r]) //s é uma posição de particionamento
      - Quicksort(A[l..s − 1])
      - Quicksort(A[s + 1..r])

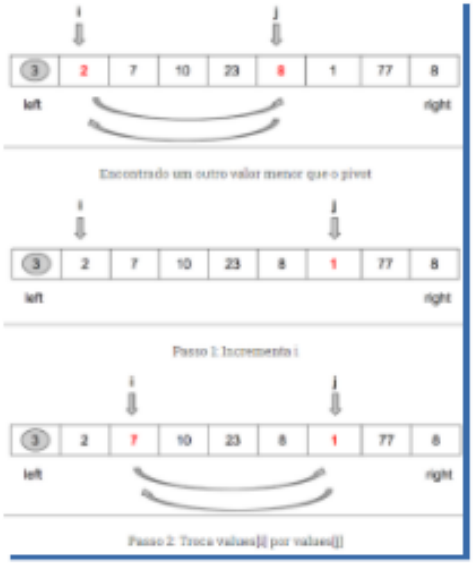
Estrutura de Dados
Ordenação

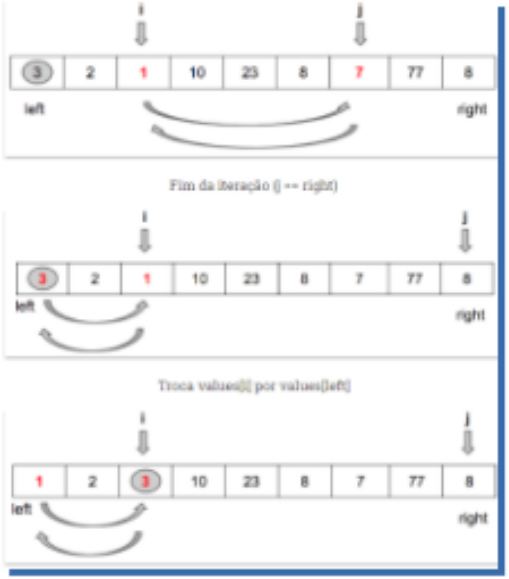
- QuickSort
  - Algoritmo de particionamento de Lomuto
    - ALGORITMO LomutoPartition(A[l..r])
    - //Subvetor de partições pelo algoritmo de Lomuto
    - // usando o primeiro elemento como pivô
    - //Entrada: Um subvetor A[l..r] do vetor A[0..n − 1],
    - // definido por sua esquerda e direita
    - // índices l e r (l ≤ r)
    - //Saída: Partição de A[l..r] e a nova posição do pivô
      - p ← A[l]
      - s ← l
      - para i ← l + 1 até r faça
        - se A[i]<p
          - s ← s + 1
          - troca(A[s], A[i])
      - troca(A[l], A[s])
      - retorna s

Estrutura de Dados
Ordenação

- Quick sort
  - Algoritmo LomutoPartition
    - Exemplo







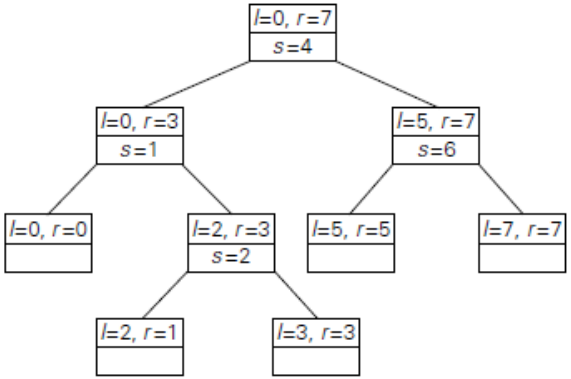
Estrutura de Dados
Ordenação

- QuickSort
  - Algoritmo de particionamento de Hoare
    - ALGORITMO PartiticionamentoHoare(A[l..r])
      - //Particiona um subarray pelo algoritmo de Hoare, usando o primeiro elemento como pivô
      - //Entrada: Subarray do array A[0..n − 1], definido por sua esquerda e direita
      - // índices l e r (l<r)
      - //Saída: Partição de A[l..r], com a posição dividida retornada como valor desta função
      - p ← A[l]
      - i←l; j ←r + 1
      - repita
        - repita i ←i + 1 até A[i]≥ p
        - repita j ←j − 1 até A[j]≤ p
        - troca(A[i], A[j])
      - até i ≥ j
      - troca(A[i], A[j]) //desfaz a última troca quando i ≥ j
      - trocar(A[l], A[j])
      - retorna j

Estrutura de Dados
Ordenação

- QuickSort
  - Algoritmo de particionamento de Hoare
    - Exemplo

0	1	2	3	4	5	6	7
	i						*j*
5	3	1	9	8	2	4	7
			i			*j*	
5	3	1	9	8	2	4	7
			i			*j*	
5	3	1	4	8	2	9	7
				i	*j*		
5	3	1	4	8	2	9	7
				i	*j*		
5	3	1	4	2	8	9	7
				j	*i*		
5	3	1	4	2	8	9	7
2	3	1	4	**5**	8	9	7
	i		*j*				
2	3	1	4				
	i	*j*					
2	3	1	4				
	i	*j*					
2	1	3	4				
	j	*i*					
2	1	3	4				
1	**2**	3	4				
1							
			i j				
		3	4				
		j	*i*				
		3	4				
			4				
						i	*j*
					8	9	7
						i	*j*
					8	7	9
						j	*i*
					8	7	9
					7	**8**	9
					7		
							9



(b)

Estrutura de Dados
Ordenação

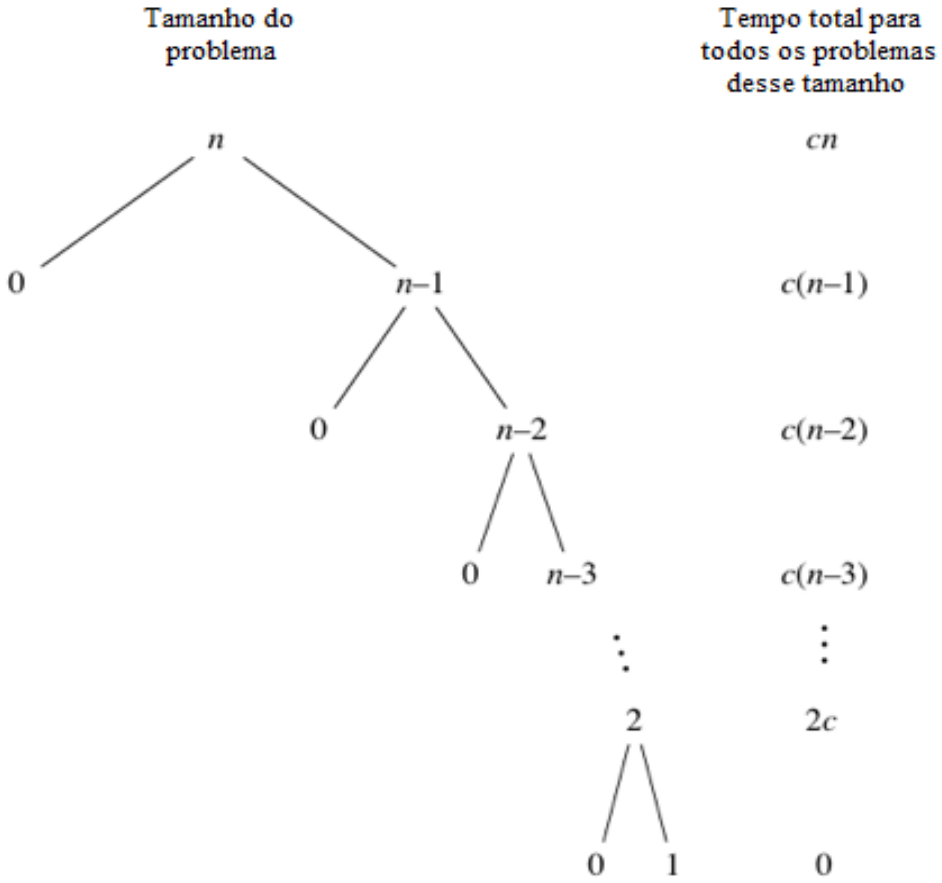
- Quick sort
  - Complexidade
    - Análise da complexidade de tempo
      - Particionamento
        - É o verdadeiro trabalho da ordenação
        - Cada elemento do vetor A é comparado uma vez com o pivô
          - A[j] pode ou não ser trocado por A[i]
        - O número total de linhas executadas por elemento no vetor é função direta de n
        - O tempo de particionamento terá complexidade O(n)
          - tempo linear de particionamento

Estrutura de Dados
Ordenação

- Quick sort
  - Complexidade
    - Análise do pior caso
      - Tamanho das partições não balanceado
        - Quando o pivô sempre é o menor ou maior elemento da partição
          - Vetor já ordenado
          - Uma das partições não terá elementos
          - A outra partição terá n-1 elementos

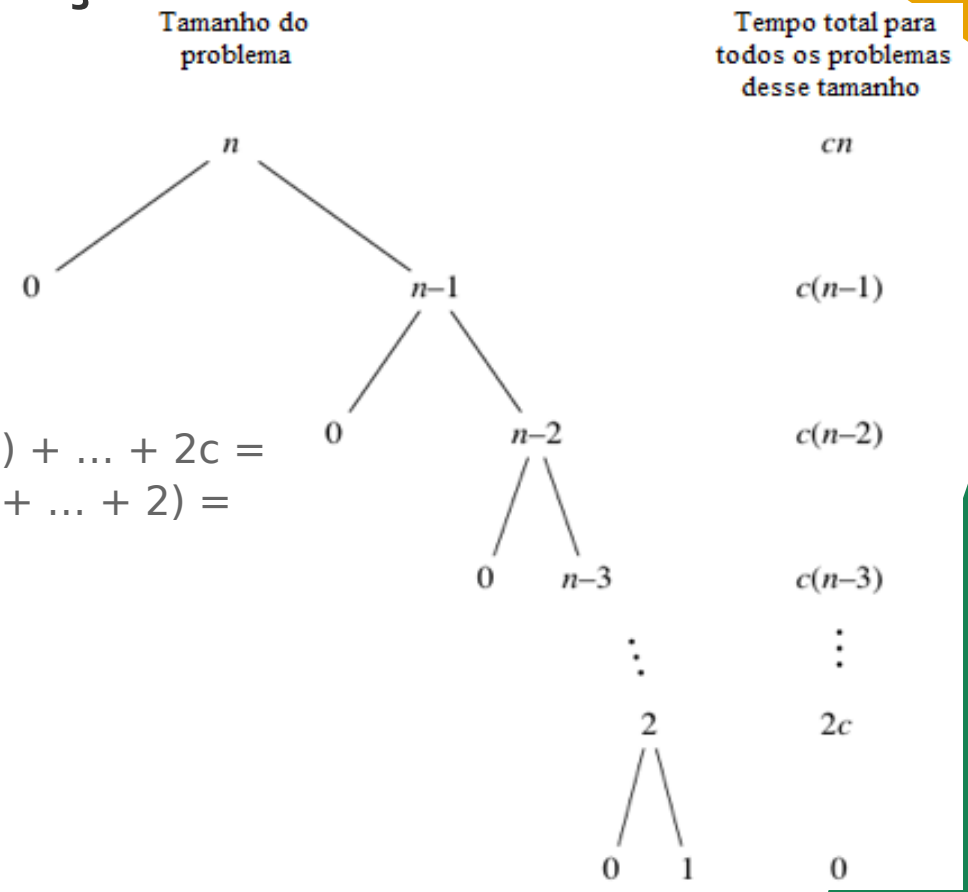
Estrutura de Dados
Ordenação

- Quick sort
  - Complexidade
    - Análise do pior caso
      - Tempo de execução



Estrutura de Dados
Ordenação

- Quick sort
  - Complexidade
    - Análise do pior caso
      - Tempo de execução
        - Somando os tempos
          - cn + c(n-1) + c(n-2) + ... + 2c =
          - c(n + (n-1) + (n-2) + ... + 2) =
          - c((n + 1)(n/2) -1)
        - Complexidade
          - $O(n^2)$

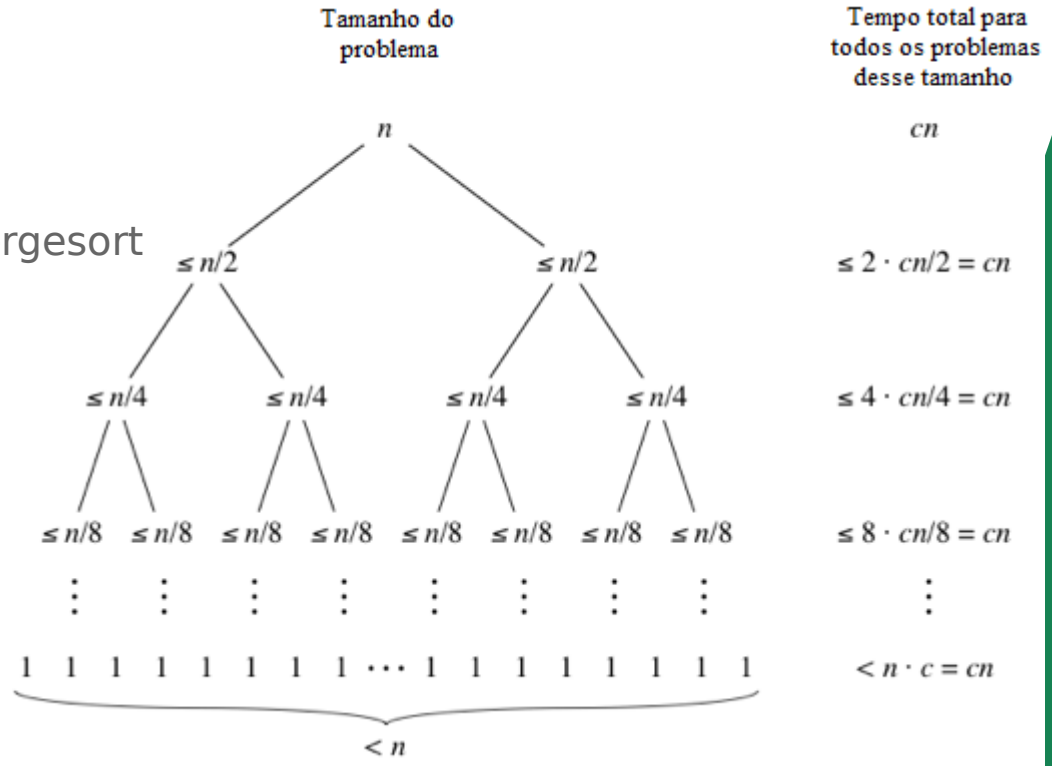


Estrutura de Dados
Ordenação

- Quick sort
  - Complexidade
    - Análise do melhor caso
      - Tamanho das partições está balanceado
        - Pivô fica no meio do vetor após o particionamento
        - Cada partição tem, no máximo, n/2 elementos

Estrutura de Dados
Ordenação

- Quick sort
  - Complexidade
    - Análise do melhor caso
      - Tempo de execução
        - Em analogia com o mergesort
          - Complexidade
            - O (n $\log_2$ n)

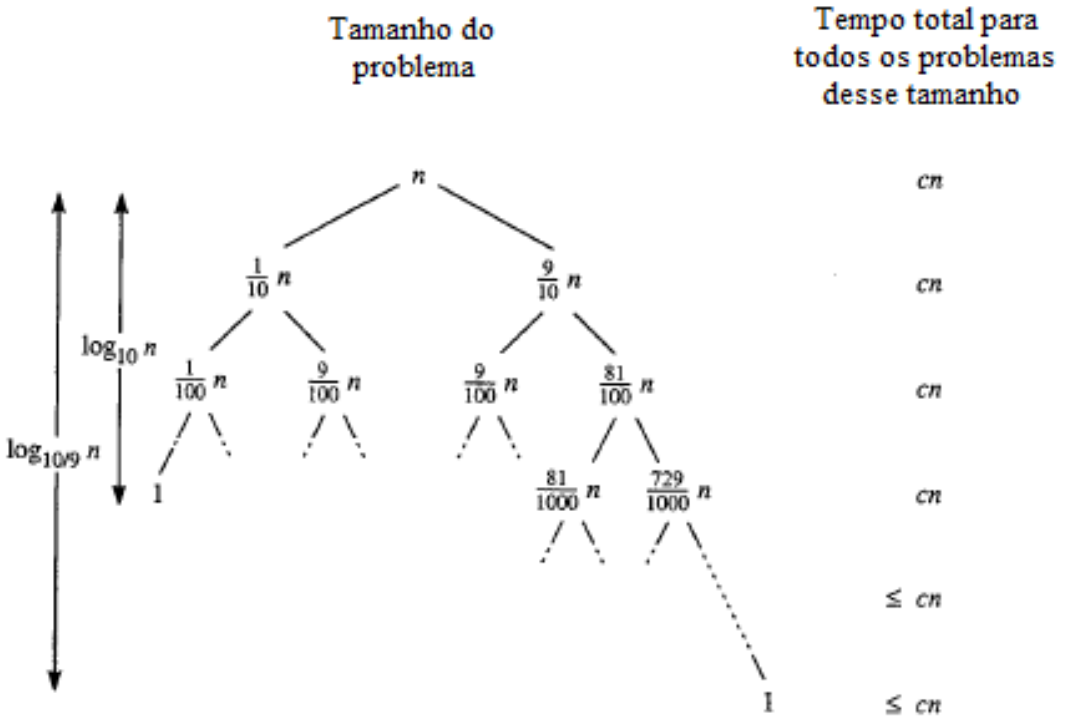


Estrutura de Dados
Ordenação

- Quick sort
  - Complexidade
    - Análise do caso médio
      - Se o particionamento é consideravelmente desbalanceado
        - O tempo se aproxima do melhor caso
      - Exemplo
        - Considerando um desbalanceamento na proporção 9:1
        - $T(n) = T(9n/10) + T(n/10) + O(n)$

Estrutura de Dados
Ordenação

- Quick sort
  - Complexidade
    - Análise do caso médio
      - Exemplo
        - Tempo de execução



Estrutura de Dados
Ordenação

- Quick sort
  - Complexidade
    - Análise do caso médio
      - Exemplo
        - Tempo de execução
          - Divisão de proporcionalidade constante
            - Produz árvore de profundidade O(log2 n)
          - O custo de cada nível será O(n)
        - Complexidade do caso médio
          - O(n log2 n)

Estrutura de Dados
Ordenação

- Quick sort
  - Complexidade
    - Análise da complexidade de espaço
      - O particionamento trabalha localmente
        - Esta partição requer espaço O(1)
      - Deve ser armazenada uma quantidade constante de informações para cada chamada recursiva aninhada
      - Como o melhor caso faz no máximo O(log n) chamadas recursivas aninhadas
        - É usado o espaço O(log n)

Estrutura de Dados
Ordenação

- Quick sort
  - Características:
    - Boa performance: O(n log n)
    - In place
    - Gasto com a recursão - media de O(log n)
    - Não stable

Estrutura de Dados
Ordenação

- Counting sort
  - Ordenação linear
  - Não utiliza comparação
  - Ordenação por contagem
  - A ideia geral é registrar a frequência dos elementos

Estrutura de Dados
Ordenação

- Counting sort
  - Passos do algoritmo
    - registrar a frequência dos elementos de A (vetor a ordenar) no vetor C (contador de frequências)
    - calcular a soma cumulativa de C
      - Esse passo registra, para cada elemento x da entrada, o número de elementos menores ou iguais a x
    - iterar sobre A do fim ao início registrando em B (vetor ordenado) o valor de A
      - Com a seguinte instrução B[C[A[i]]] = A[i]
      - Decrementando C[A[i]]

Estrutura de Dados
Ordenação

- Counting sort
  - Algoritmo

Algorithm 1 Counting-Sort(A,B,n,k)

```
for i = 0 to k do
    C[i] = 0;
end for
for j = 1 to n do
    C[A[j]] = C[A[j]] + 1; {C[i] é o número de j's tais que A[j]=i}
end for
for i = 1 to k do
    C[i] = C[i] + C[i − 1]; {C[i] é número de j's tais que A[j] ≤ i}
end for
for i = n decrescendo até 1 do
    B[C[A[j]]] = A[j];
    C[A[j]] = C[A[j]] − 1;
end for
```

Estrutura de Dados
Ordenação

- Counting sort
  - Exemplo
    - Ordenar [1, 9, 1, 3, 4, 7, 6]
      - n = 7
      - k = 9

Estrutura de Dados
Ordenação

- Counting sort
  - Complexidade
    - Complexidade de tempo
      - O Counting Sort tem em sua implementação 3 laços principais.
        - O primeiro percorre o vetor A (tamanho n)
        - O segundo percorre o vetor C (tamanho k)
        - O terceiro percorre novamente o vetor A
      - Assim, temos T(n)=2n+k
        - Aplicando as diretrizes para análise assintótica: T(n)=(n+k)
        - O(n+k)
    - Complexidade de espaço
      - Vetor de contagem C (tamanho k)
      - Vetor de resultado B (tamanho n)
      - O(n+k)

Estrutura de Dados
Ordenação

- Counting sort
  - Considerações
    - Para k muito maior que n
    - Exemplo:
      - A=[1,3,2,1,9874392]
      - O vetor de contagem C tem que ter tamanho=9874392
        - Mesmo tendo que ordenar apenas 5 elementos
        - Seria muito ruim

Estrutura de Dados
Ordenação

- Counting sort
  - Considerações
    - Para k muito menor que n
    - Exemplo
      - A = [1, 3, 2, 1, 1, 5, 3, 2, 5, 4, 2, 1, 2, 1, 1, 2, 1, 4, 5, 2, 2, 3, 2]
      - O vetor de contagem C tem que ter tamanho=5
        - Para ordenar um vetor com 23 elementos
        - Isso representa uma vantagem

Estrutura de Dados
Ordenação

- Counting sort
  - Considerações
    - O Counting Sort baseia-se na ideia de que um valor inteiro pode ser mapeado para o índice de mesmo valor em um vetor auxiliar
    - Essa estratégia impede, em um primeiro momento, de ordenar uma sequência com números negativos
    - Além disso, na implementação inicial foram excluídos elementos iguais a zero
    - É possível fazer algumas mudanças simples no Counting Sort para que o mesmo passe a também ordenar sequências com esses valores
    - A ideia é simples: basta identificarmos o menor elemento do vetor (menor) e usar esse valor como um “salto” para adicionar os elementos
    - É uma ideia similar a fazer um shift para a direita em todos os elementos
    - O menor elemento vetor tem sua frequência registrada na posição zero

Estrutura de Dados
Ordenação

- Counting sort
  - Características
    - Fácil de implementar
    - Fácil de entender
    - Stable
    - Não in place
    - Assume que os elementos estão distribuídos em um intervalo pequeno (1..k)
    - Não faz comparações
      - Usa o índice do vetor

Estrutura de Dados
Ordenação

- Radix sort
  - Consiste em ordenar um vetor A[1 . . . n] de inteiros quando se sabe que todos os inteiros podem ser representados com apenas d dígitos, onde d é uma constante
  - Por exemplo, os elementos de A podem ser CEPs, ou seja, inteiros de 8 dígitos

Estrutura de Dados
Ordenação

- Radix sort
  - É possível ordenar os elementos do vetor, dígito a dígito, da seguinte forma
    - São separados os elementos do vetor em grupos que compartilham o mesmo dígito mais significativo
    - Em seguida, são ordenados os elementos em cada grupo pelo mesmo método, levando em consideração apenas os $d - 1$ dígitos menos significativos
  - Esse método funciona, mas requer o uso de bastante memória adicional para a organização dos grupos e subgrupos

Estrutura de Dados
Ordenação

- Radix sort
  - É possível evitar o uso excessivo de memoria adicional começando pelo dígito menos significativo
  - Para que Radix Sort funcione corretamente, ele deve usar um método de ordenação estável
    - Por exemplo, o Counting sort

Estrutura de Dados
Ordenação

- Radix sort
  - Algoritmo
    - Suponha que os elementos do vetor A a ser ordenado sejam números inteiros de até d dígitos. O Radix Sort é simplesmente

```
RADIX-SORT(A, n, d)
para i ← 1 até d faça
    Ordene A[1 . . . n] pelo i-ésimo dígito
    usando um método estável
```

Estrutura de Dados
Ordenação

- Radix sort
  - Exemplo

329		720		720		329
457		355		329		355
657		436		436		436
839	→	457	→	839	→	457
436		657		355		657
720		329		457		720
355		839		657		839
		↑		↑		↑

Estrutura de Dados
Ordenação

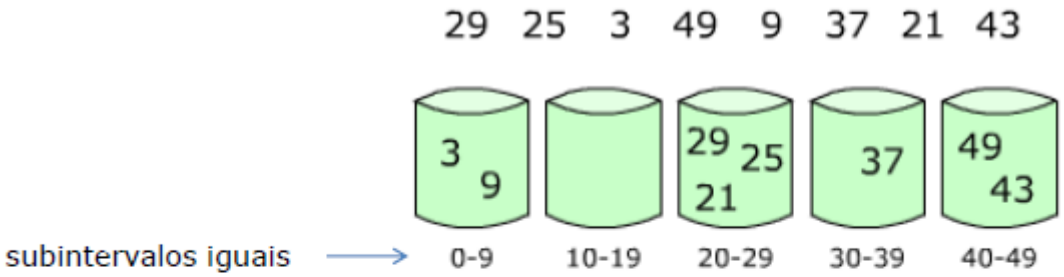
- Radix sort
  - Complexidade
    - A complexidade do Radix Sort depende do algoritmo auxiliar utilizado
    - RadixSort(n) ϵ O(d*OrdenadorAuxiliar(n))
      - Onde d é a quantidade de dígitos dos números (ou quantidade de caracteres das strings) que serão ordenados

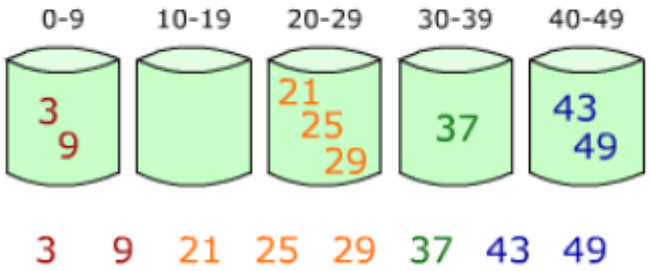
Estrutura de Dados
Ordenação

- Radix sort
  - Características
    - Stable
    - O(nd + kd)
      - Counting Sort
        - k: domínio dos possíveis valores
        - n: número de elementos
        - d: número de dígitos
      - Bom quando d for bem menor do que n: O(n)
      - Não é in place
        - Precisa de memoria extra O(n)
    - Particularmente eficiente quando se tem muitos números a ordenar, mas as chaves são de tamanho pequeno

Estrutura de Dados
Ordenação

- Bucket sort
  - Intuição





Estrutura de Dados
Ordenação

- Bucket sort
  - Lógica
    - Inicializar um vetor de "baldes", inicialmente vazios
    - Incluir cada elemento do vetor original em um balde
      - Ordenar todos os baldes não vazios
      - Ou inserir no balde ordenado (insertiont sort)
    - Colocar os elementos dos baldes que não estão vazios no vetor original
      - Concatenando os baldes em ordem

Estrutura de Dados
Ordenação

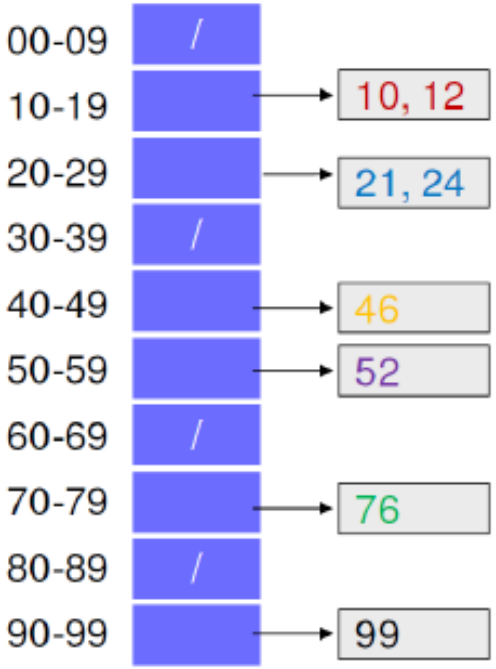
- Bucket sort
  - Algoritmo

```
BUCKET-SORT(A) {
    buckets ← new array of 10 empty lists
    for i ← 1 to A.length {
        insert A[i] into buckets[A[i]/10]
    }
    for i ← 0 to 9 {
        next-sort(buckets[i])
    }
    return <buckets[0]>; ... ;<buckets[9]>
}
```

Estrutura de Dados
Ordenação

- Bucket sort
  - Exemplo

Exemplo:
A = [10,21,24,12,46,99,52,76]



Estrutura de Dados
Ordenação

- Bucket sort
  - Complexidade

Analisando o algoritmo...

```
BUCKET-SORT(A) {
    buckets ← new array of 10 empty lists
    for i ← 1 to A.length {                    → n
        insert A[i] into buckets[A[i]/10]
    }
    for i ← 0 to 9 {                           → buckets*O(f(n))
        next-sort(buckets[i])
    }
    return <buckets[0]>; ... ;<buckets[9]>
}
```

$$\Theta(m+n)$$

Estrutura de Dados
Ordenação

- Bucket sort
  - Características
    - Stable
      - Varrendo os elementos da esquerda para a direita inserindo nos buckets
    - Não é in place
      - O(m + n)
        - m: faixa de valores a serem ordenados
        - n: tamanho do vetor
    - Assume que os elementos estão uniformemente distribuídos
      - Senão, todos os elementos caem em um único bucket, no pior caso

Estrutura de Dados
Ordenação

- Bucket sort
  - Comparação

Algoritmo	Caso Médio	Pior Caso	Espaço	*Stable*
Bubble Sort	$O(n^2)$	$O(n^2)$	*In place*	Sim
Selection Sort	$O(n^2)$	$O(n^2)$	*In place*	Sim
Insertion Sort	$O(n^2)$	$O(n^2)$	*In place*	Sim
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n)$	Sim
Quicksort	$O(n \log n)$	$O(n^2)$	Tamanho da pilha de recursão. $O(\log n)$	Não
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	Sim
Radix Sort	$O(nd+kd)$	$O(nd+kd)$	$O(n)$	Sim
Bucket Sort	$O(m+n)$	$O(m+n)$	$O(m+n)$	Sim